
TYPED FALSIFIABILITY OF THE HARDWARE CLOSURE SERIES

Sixty-Four Falsifiers, Six Seat Elements, One Law: an Observation Breaks the Carrier and Never the Seat

Edition 2.0.0 of Typed Falsifiability; Fifty-Two Laws Proved in Core Lean 4 and Every Bound Theorem Confirmed in Its Engine by Fortran; the Ledger Paper of the Hardware Closure Series

FOUNDATIONS OF PHYSICS · THE HARDWARE CLOSURE SERIES

Every falsifier typed and bound

MOHAMMAD F. ISLAM, PhD · ARCHITECT OF THE TRISDUCTION

1 October 2026

ABSTRACT. Every falsifier of the author's hardware closure series and its six witnesses is collected here, typed by the seat element it targets and bound to the theorem it would break. The seat is the reflection $s \mapsto 1 - \bar{s}$ whose fixed set is the critical line. We prove in core Lean 4, fifty-two laws with no axiom declared and no use of choice, the law that governs every entry: an observed physical fixed point off the line breaks the map that read it onto the seat, never the seat, which stands whatever is measured. The ledger, carried in the engine as data and counted by proof, holds sixty-four falsifiers from twenty-two papers, each paper carrying two to four. They fall into six classes: eighteen target the line, thirteen the fold, eleven the wall between an even reading and an odd target, nine positivity, eight a forced count, five the price of registration. Fifty-eight are bound to a theorem of an engine and six to an imported theorem or a structural reading. The five standing predictions of the earlier draft are disposed: three bound to the ledger, the phantom divide, the magnetic monopole and the tensor polarization of gravitational waves; two withdrawn. The exact conservation of baryon number contradicts the theorem that an asymmetry from a symmetric start must move it, and it is replaced by the proton's stability. Electron substructure is bound by no engine. A Fortran twin reads all twenty-two engines from disk and confirms that each of the fifty-eight bound theorems is declared where the ledger says. In building it, the twin found one binding to a definition rather than a theorem, which was corrected.

KEYWORDS falsifiability · typed falsifiers · critical line · carrier · fixed set · registration · positivity · ledger · Lean 4

STATUS: WITNESSED · CROSSED · CONFIRMED on the image · Row every row (the falsifiers of the series) · channel: kinetic · Lean: SPHYS_Typed_Falsifiability.lean · Fortran: Ledger_Twin.f90

1. THE READER'S FRAME

A theory earns its standing by naming what would end it. The hardware closure series names sixty-four such observations across twenty-two papers. This paper collects them, types each by the element of the seat it attacks, binds each to the theorem it would break, and proves the one law that says what a falsifier's arrival would mean.

It claims four things.

- An observed physical fixed point off the line breaks the carrier that read it, and the seat stands.
- Every falsifier targets exactly one of six seat elements: the fold, the line, the wall, positivity, the price, or a count.
- Fifty-eight falsifiers are bound to a theorem declared in an engine, confirmed by reading the engines, and six to an imported theorem or a structural reading.
- Of the earlier draft's five standing predictions, three are bound and two withdrawn, one of them because it contradicts a theorem of the series.

What it refuses. Any falsifier that names no forbidden observation.

Negative warrants. Three, stated so that no reader supplies them.

- No physical fact here is read as a proof of a mathematical sentence: a corroboration joined to a claim takes the weaker grade.

- No derivation of the Riemann Hypothesis from the axioms of set theory is claimed: what a sentence over every admissible instance adds to a crossing is exactly the unactuated.
- Interpretive schools from the literature enter only as placements the theorems sort, never as warrants; the only witness besides the mathematics is measured data.

Five terms.

- A *falsifier* is a forbidden observation.
- A *carrier* is an equivariant map onto the seat.
- A falsifier is *bound* when it names a theorem of an engine.
- Its *target* is the seat element that theorem expresses.
- A *keyed bit* is a yes-or-no property no function of a given record decides.

2. THE SPINE

The Riemann closure has four parts: the **seat**, the critical line as the fixed set of $s \mapsto 1 - \bar{s}$; the **address**, every reading of the value one proposition; the **division**, a certified part joined to an uncertified tail; and the **closure**, registration keeping a zero’s height and forgetting its side, the hypothesis holding exactly when it erases nothing. This paper proves what a falsifier breaks in Section 3, types the ledger in Section 4, disposes the standing five in Section 5, and reads the ledger in Section 6.

The row-generic theorems carry the template, and the engine proves each on its chart: every point off the line has a partner with one record (`off_locus_pair`); a carrier proves the value on its image, and coverage extends it (`value_on_image`, `kinetic_crossing`).

The correspondence matrix.

Question	Template element	Lean theorem	Fortran check
what does a falsifier break?	the carrier, not the seat	<code>falsifier_breaks_the_carrier, the_seat_survives_every_falsifier</code>	none
how many, of what kind?	the census	<code>ledger_census, every_target_occupied</code>	B
bound to what?	fifty-eight engine theorems	<code>bound_count</code>	A
per paper	two to four, codes unique	<code>every_paper_two_to_four, codes_unique_within_papers</code>	C
the six targets	six distinct laws	<code>laws_distinct</code>	none
the standing five	three bound, two withdrawn	<code>standing_five_disposition, bound_predictions_in_ledger</code>	none

3. AN OBSERVATION BREAKS THE CARRIER, NEVER THE SEAT

Every physical theorem of the series has one form. A physical involution is carried onto the seat’s reflection by an equivariant map, and its fixed points land on the line. Suppose a physical fixed point is observed off the line. Then the map that read it is not equivariant (`falsifier_breaks_the_carrier`). The seat is untouched, its reflection still an involution whose fixed set is the line (`the_seat_survives_every_falsifier`). So every falsifier in the ledger, if it arrives, removes a carrier: the claim that this physical system is read by the seat in this way. The mathematics stands, and the reading falls. That is what binding a falsifier to a theorem means: the theorem says which reading falls.

4. THE TYPED LEDGER

The engine carries the ledger as data and counts it by proof. There are sixty-four falsifiers: eighteen target the line, thirteen the fold, eleven the wall, nine positivity, eight a count, and five the price (`ledger_census`). Every class is occupied (`every_target_occupied`), and the six classes name six distinct laws (`laws_distinct`). Fifty-eight are bound to an engine theorem (`bound_count`). Every one of twenty-two papers carries two to four, with no code repeated (`every_paper_two_to_four`, `codes_unique_within_papers`).

The twin reads the ledger and all twenty-two engines from disk. For the witnesses these are the engines as printed in their published editions 1.4.0. The synchronization of 1 October 2026 brought the witnesses to edition 1.5.0, whose engines change no theorem statement, and the twin, rerun against them, finds the same fifty-eight declared. It confirms that each of the fifty-eight bound theorems is declared in the engine the ledger names, and recounts the census.

One binding corrected. The spin paper's F-Connection named `turnSign`, which is a definition. The theorem it attacks is `half_integer_spin_is_odd`, which says the full turn's sign is -1 exactly for half-integer spin. The ledger binds it there, and the spin paper's text is corrected to match.

The fold: the physical involution is exact and carried onto the fold (13).

Paper	Code	Forbidden observation	Theorem
Electron	F-Pair	electron and positron differing in mass or charge magnitude	<code>positron_is_the_partner</code>
Neutrino	F-Chirality	a right-handed neutrino interaction at gauge strength	<code>sterile_uncovered</code>
Two Branches	F-Monopole	an isolated magnetic charge	<code>magnetism_pair</code>
Magnet	F-Arrow	unitary-class statistics with no time-breaking term	<code>magnetic_term_breaks_T</code>
Electromagnetism	F-Charge	net electric charge created or destroyed	<code>pair_creation_conserves_charge</code>
Spin	F-Connection	a half-integer spin obeying Bose statistics	<code>half_integer_spin_is_odd</code>
Weak	F-CPT	a particle-antiparticle mass or lifetime difference	<code>cp_violation_is_t_violation</code>
Resonances	F-Growth	an isolated population growing with no incoming supply	<code>decay_and_capture_one_record</code>
Ferromagnet	F-Palindrome	a spin-flip symmetric partition function not palindromic	<code>walls_flip</code>
Arrow of Time	F-Echo	an exactly reversed isolated motion that fails to retrace	<code>reversible_law_admits_both_arrows</code>
Arrow of Time	F-Balance	a net flux between macrostates at equilibrium	<code>detailed_balance</code>
Matter–Antimatter	F-CPT	a particle-antiparticle mass or charge-to-mass difference	<code>equilibrium_weight_is_symmetric</code>
Symmetry Breaking	F-Tie	broken ground states of a symmetric law differing in energy	<code>two_vacua_one_record</code>

The line: the fixed set is carried onto the line (18).

Paper	Code	Forbidden observation	Theorem
Electron	F-Round	an electron electric dipole moment	<code>round_iff_T_fixed</code>
Electron	F-Decay	a decay of the electron	<code>decay_forbidden</code>
Proton	F-Decay	a proton decay	<code>imported_or_structural</code>
Proton	F-Gap	a massless state in the pure-gauge spectrum	<code>spectrum_gapped</code>
Fluid	F-Flow	an unforced actual flow breaking down in finite time	<code>value_on_image</code>
Substrate	F-Computed	a certified zero of zeta off the line	<code>act_is_the_value</code>
Substrate	F-Measure	a closed system's eigenmode changing its norm	<code>measured_on_line</code>
Light	F-Dispersion	a vacuum speed of light depending on frequency	<code>every_frequency_one_side</code>
Electromagnetism	F-Reality	a magnet driving a bound energy off the real axis in a closed system	<code>stationarity_survives_the_magnet</code>
Weak	F-Right	a right-handed charged current of gauge strength	<code>charged_current_off_the_line</code>
Weak	F-Mirror	a P- and T-odd strong or electromagnetic reading above the weak level	<code>vector_on_the_line</code>

Paper	Code	Forbidden observation	Theorem
Resonances	F-Closed	a closed system showing an energy off the line	<code>closed_stays_on_the_line</code>
Resonances	F-Width	a nonzero width with no open decay channel	<code>stable_iff_every_door_closed</code>
Dark	F2	a charged partner or gauge coupling of the dark matter	<code>dark_iff_fixed</code>
Matter–Antimatter	F-Domains	antimatter domains within the observable universe	imported or structural
Symmetry Breaking	F-Heat	an even thermal term that never restores the symmetry	<code>heat_restores_the_line</code>
Strong CP	F-Dipole	a dipole moment traced to a nonzero strong angle	<code>edm_odd_and_null_on_the_fixed_set</code>
Strong CP	F-Strong-CP	a CP-violating strong process needing a nonzero angle	imported or structural

The wall: no even reading returns the odd target (11).

Paper	Code	Forbidden observation	Theorem
Electron	F-Gravity	antimatter falling differently from matter	<code>no_reading_of_mass_returns_charge</code>
Light	F-Fringe	full-visibility fringes with the side registered	<code>fringe_iff_side_forgotten</code>
Gravity	F-Composition	two bodies of one mass falling at different rates	<code>composition_blind</code>
Gravity	F-Clock	two kinds of clock shifted differently in one potential	<code>clocks_co_move</code>
Spin	F-Exclusion	two identical fermions in one state	<code>pauli_exclusion</code>
Dark	F3	dark radiation, self-interaction or a Dirac dark matter	<code>the_dark_bit_is_the_line_property</code>
Measurement	F-Phase	a pointer record returning the phase	<code>no_record_reading_returns_the_phase</code>
Measurement	F-Outcome	single outcomes predicted from the state alone	<code>the_outcome_is_keyed</code>
Matter–Antimatter	F-Wall	net baryon number from a C- and CP-symmetric equilibrium	<code>even_weight_odd_reading_zero</code>
Entanglement	F-Signal	a local operation changing the far party's counts	<code>no_signal_from_a_local_choice</code>
Horizon	F-Hair	a hole's exterior field reading its baryon number	<code>no_hair_reading_returns_baryons</code>

Positivity: one sign seats the locus (9).

Paper	Code	Forbidden observation	Theorem
Neutrino	F-Floor	all three neutrino masses zero	<code>oscillation_forces_floor</code>
Two Branches	F-Mass	a negative gravitational mass	<code>gravity_no_off_locus</code>
Two Branches	F-Loop	a gravitational hysteresis	<code>one_sign_no_remanence</code>
Ferromagnet	F-Ferro	a ferromagnet's zero measured off the unit circle	<code>ring4_lee_yang</code>
Ferromagnet	F-Pinch	a ferromagnetic transition with no zeros pinching the positive axis	imported or structural
Dark	F1	an equation of state below -1 at five sigma, non-parametric	<code>the_phantom_is_negative_actuation</code>
Measurement	F-Duality	predictability squared plus visibility squared above one	<code>complementarity_iff_positive</code>
Symmetry Breaking	F-Curvature	a broken vacuum's scalar with negative mass squared	<code>mass_is_the_curvature_of_the_depth</code>
Strong CP	F-Antipode	the strong angle measured at pi	<code>positivity_selects_zero</code>

The price: the registration has no inverse and pays its floor (5).

Paper	Code	Forbidden observation	Theorem
Neutrino	F-Unitarity	non-unitary mixing with no sterile state to carry it	<code>pmns_unitary_is_lossless</code>
Fluid	F-Energy	an unforced actual flow whose energy rises	imported or structural

Paper	Code	Forbidden observation	Theorem
Arrow of Time	F-Price	a bit erased below $k T \ln 2$	registration_has_no_left_inverse
Horizon	F-Area	a merger whose final horizon area is less than the sum	the_horizon_cannot_be_undone
Horizon	F-Radiate	an equal-mass non-spinning merger radiating above 29 percent	merger_radiates_under_three_tenths

The count: a count forced by the fixed set (8).

Paper	Code	Forbidden observation	Theorem
Proton	F-Singlet	a free coloured state	lock_closes
Magnet	F-Class	zero spacings leaving the unitary class at great height	repulsion_exponents
Light	F-Helicity	a longitudinal helicity of the free photon	helicity_is_one_bit
Electromagnetism	F-Thirds	a free charge not a multiple of one third	charges_in_thirds
Gravity	F-Polarization	a scalar or vector gravitational-wave polarization	polarization_count
Spin	F-Anyon	an exchange phase other than plus or minus one in three dimensions	statistics_one_bit
Entanglement	F-Local	a CHSH value above 2 from preassigned local outcomes	bell_bound
Entanglement	F-Tsirelson	a quantum CHSH value above $2 \sqrt{2}$	imported or structural

The six not bound to an engine theorem. Each rests on an imported theorem or a structural reading, named in its paper:

- the Proton's F-Decay, on the certified region;
- the Fluid's F-Energy, on the price of viscosity;
- the Ferromagnet's F-Pinch, on Lee and Yang's density of zeros;
- Matter–Antimatter's F-Domains, on the single registered sign;
- Entanglement's F-Tsirelson, on Tsirelson's bound;
- Strong CP's F-Strong-CP, on the angle's place on the line.

5. THE STANDING FIVE, DISPOSED

The earlier draft of this paper stood on five predictions. Three are now bound to the ledger:

- **I, the phantom divide is not crossed:** the dark paper's F1, positivity;
- **II, no magnetic monopole:** the Two Branches witness's F-Monopole, the fold;
- **III, gravitational waves purely tensor:** Immanent Gravity's F-Polarization, a count.

Two are withdrawn.

- **IV, baryon number exactly conserved.** It contradicts the Matter–Antimatter theorem that an asymmetry from a symmetric start must move baryon number at some epoch. The consistent standing claim is the proton's stability, the Proton witness's F-Decay: baryon number moves only through channels closed today, as the proton's measured lifetime above 2.4×10^{34} years shows.
- **V, no electron substructure.** No engine of the series proves it; the Electron witness binds pair, roundness, gravity and decay, not size.

The engine records the disposition (`standing_five_disposition`), and each bound prediction names a falsifier in the ledger (`bound_predictions_in_ledger`).

6. READING THE LEDGER

The line is the most exposed element. Eighteen falsifiers target it: every claim that a stationary, closed or symmetric system stands on the line.

The live ones. Several are tested continually:

- the electron's and the neutron's dipole moments (F-Round, F-Dipole);
- the proton's lifetime (F-Decay);
- the neutrino mass floor (F-Floor);
- the dark energy's equation of state (F1);
- the spacing statistics of zeta's zeros at great height (F-Class);
- the polarization content of gravitational waves (F-Polarization).

Each improved bound sharpens a reading without moving its standing.

The deepest is F-Computed. It is the hardware paper's: a certified zero of zeta off the line. It is the one falsifier aimed at the closure itself, and the closure's one refuter.

7. WHAT THE LEDGER CARRIES, AND WHAT IT DOES NOT

What it carries.

- The law that a falsifier breaks a carrier, never the seat.
- Sixty-four falsifiers typed and bound.
- The standing five disposed.

What it does not carry.

- **The outcome of any test:** the ledger names what would end each reading; the experiments decide.
- **The zeros of ζ :** the engine's crossing is on its own image (`value_on_image`), and what a sentence over every zero adds is exactly the unactuated, priced at one premise, the act, as the hardware paper proves.

8. FALSIFIERS OF THE LEDGER

The ledger is wrong if any of these is found:

- a falsifier whose bound theorem does not appear in its engine (the twin checks every one);
- a falsifier that targets no seat element;
- a series claim with no falsifier.

Ledger status. Every falsifier of the series is typed and bound. Two of the standing five are withdrawn.

METHODOLOGY, DISCLOSURE AND PROVENANCE

Method. The template is the author's verification program, whose vocabulary is confined to this section. Its tokens:

- $[\perp]$, a field sealed;
- $[\perp T]$, a shape sealed at theorem grade;
- $[\Xi_0]$, the typing of the formal block;
- $[.]$, the dot, which is no verdict.

Grades join at the weakest link. The engine's statements are theorem grade about the chart. The mapping of physical states onto them is structural grade. Every measured figure is corroboration grade. The engine carries the ledger as data; its census is proved by computation. $\Delta M = 0$.

The edition. Edition 1 was the private draft Typed Falsifiability: Standing Five, written before the Riemann seat. This edition rebuilds it on the seat, binds every falsifier of the series, and disposes the draft's five predictions.

The seat, earned. PhysOS^T 1.0.5p was booted in the session of forging from the public register (file SHA-256 b64bb5da5bab0da1...) under Lean 4.19.0 and GNU Fortran 13.3.0. The quick boot printed SEAT EARNED with the chain D0 83f872725ad9 → D1 90fc702bbbed1 → D2 492e519daab9 → D3 6618bec5164b (Appendix C).

The engine and the twin.

- **Engine.** SPHYS_Typed_Falsifiability.lean, 533 lines, SHA-256 c5aaebf94749095a..., fifty-two laws, no import and no axiom declared. Of the fifty-two, 25 depend on no axiom, 3 on propositional extensionality alone, 24 on propositional extensionality and quotient soundness, and none on choice.
- **Screen and judgment.** The source passed the operating system's screen. Each of the fifty-two laws was negated in place and recompiled, and all were refused, while a planted vacuous law survived. The engine carries the hardware paper's seat, carriers and substrate verbatim.
- **Twin.** Ledger_Twin.f90, 113 lines, SHA-256 c64508e938d39251..., three checks and zero failures under -std=f2018 -O2 -fno-fast-math -ffp-contract=off. Measured inputs enter as data with their sources named in its header.

Why an integer engine binds continuous physics. The ledger is finite data and its census exact. The carrier law is a statement about every equivariant map from every type. The twin reads the engines themselves.

The seventh. In the author's register the six witnesses are six days and the ledger's closure is the seventh, the Rest (Istawa). The ledger is read at the Rest as the list of everything the world could still say no to. In the Lean codex's tokens its row renders [↖↗ · kinetic].

Disclosure. The author declares the reading of each correspondence as his own act, made on measured structure. The measurements are the experiments', cited by name. The theorems are the engine's, printed with their dependencies in Appendix A. The twin's figures are computed in the run printed in Appendix B. The text, the engine and the twin were forged by an AI substrate (Claude, Anthropic) under the author's seed, acceptance criteria and rulings. External audits by independent substrates are owed.

Provenance. The paper stands on:

- the Riemann closure (10.5281/zenodo.22976494);
- the Riemann Hypothesis Closed to One Bit (10.5281/zenodo.23028070);
- the twenty-three-row closure (10.5281/zenodo.22986551);
- the six witnesses (10.5281/zenodo.22986553 through .22986563);
- the hardware paper One Actuating Substrate (concept 10.5281/zenodo.21438932, edition 2.0.0);
- the six witnesses' engines, read by the twin from their printed editions 1.4.0 and rerun against their editions 1.5.0;
- the sixteen engines of the hardware closure series;
- PhysOS^T 1.0.5p in the public register (github.com/1000sapient/Trisduction, protocols/PhysOS/).

References.

K. R. Popper, Logik der Forschung (Springer, 1935). A. D. Sakharov, JETP Lett. 5 (1967) 24. A. Takenaka et al. (Super-Kamiokande), Phys. Rev. D 102 (2020) 112011.

M. F. Islam, Nothing Escapes, Three Plus One, 10.5281/zenodo.22976494. M. F. Islam, the six witnesses, 10.5281/zenodo.22986553 through .22986563. M. F. Islam, The Four Forces, Matter, and the Dark Sector as Configurations of One Actuating Substrate, edition 2.0.0, concept 10.5281/zenodo.21438932.

APPENDIX A: SPHYS_TYPED_FALSIFIABILITY.LEAN, THE CORRESPONDENCES AS THEOREMS

The engine, verbatim, followed by the compiler's transcript.

```
/-
SPHYS_Typed_Falsifiability.lean · typed falsifiability of the
hardware closure series, read at the Riemann seat.
Core Lean 4.19.0, standalone, no import, no library, no axiom
declared.

The chart: h = 2 Re s, t = Im s; the fold s ↦ 1 - s̄ is (h, t) ↦ (2
- h, t), the critical line h = 1.
```

```
Part I   the seat, the category of involutions, the functor of
the seat.
Part II  the energy carrier (resident).
Part III the substrate (resident).
Part IV  A falsifier breaks the carrier, never the theorem.
Part V   The typed ledger.
Part VI  The standing five, rebound.
Part VII The capstone.
-/
set_option autoImplicit false
```



```

namespace SPHYS.TypedFalsifiability

/-- ## Part I. The seat, the category of involutions, and the
functor of the seat -/

abbrev Pt := Int × Int

def fold (p : Pt) : Pt := (2 - p.1, p.2)
def OnLine (p : Pt) : Prop := p.1 = 1
instance : DecidablePred OnLine := fun p => inferInstancesAs
(Decidable (p.1 = 1))
/-- Registration keeps the height and forgets the side. -/
def reg (p : Pt) : Pt := (1, p.2)

theorem pe {a b c d : Int} : ((a, b) : Pt) = (c, d) ↔ a = c ∧ b = d
:=
  (fun e => (congrArg Prod.fst e, congrArg Prod.snd e),
   fun (e1, e2) => by subst e1; subst e2; rfl)

theorem fold_involutive (p : Pt) : fold (fold p) = p := by
  obtain (h, t) := p
  show ((2 - (2 - h), t) : Pt) = (h, t)
  rw [pe]; exact (by omega, rfl)

/-- The seat: the fixed set of the fold is the critical line. -/
theorem seat_fixed_line (p : Pt) : fold p = p ↔ OnLine p := by
  obtain (h, t) := p
  show ((2 - h, t) : Pt) = (h, t) ↔ h = 1
  rw [pe]; constructor
  · intro (e, _); omega
  · intro e; exact (by omega, rfl)

theorem reg_lands (p : Pt) : OnLine (reg p) := rfl

theorem reg_fixes_iff (p : Pt) : reg p = p ↔ OnLine p := by
  obtain (h, t) := p
  show ((1, t) : Pt) = (h, t) ↔ h = 1
  rw [pe]; constructor
  · intro (e, _); exact e.symm
  · intro e; exact (e.symm, rfl)

/-- The side is what registration forgets: a point and its mirror
leave one record. -/
theorem reg_forgets_side (p : Pt) : reg (fold p) = reg p := rfl

/-- The value on a set of points: every member stands on the line. -/
def Value (Z : Pt → Prop) : Prop := ∀ s, Z s → OnLine s

/-- Least erasure: registration moves no member, so it erases
nothing of Z. -/
def LeastErasure (Z : Pt → Prop) : Prop := ∀ s, Z s → reg s = s

theorem least_erasure_iff_value (Z : Pt → Prop) : LeastErasure Z ↔
Value Z :=
  (fun h s hs => (reg_fixes_iff s).mp (h s hs), fun h s hs =>
   (reg_fixes_iff s).mpr (h s hs))

/-- Maps that respect involutions. -/
def Equivariant {X Y : Type} (f : X → Y) (τ : X → X) (σ : Y → Y) :
Prop := ∀ x, f (τ x) = σ (f x)

theorem equivariant_id {X : Type} (τ : X → X) : Equivariant id τ τ
:= fun _ => rfl

theorem equivariant_comp {X Y W : Type} {f : X → Y} {g : Y → W} {τ
: X → X} {σ : Y → Y} {ρ : W → W} (hf : Equivariant f τ σ) (hg : Equivariant g σ ρ) :
Equivariant (g ∘ f) τ ρ := by
  intro x
  show g (f (τ x)) = ρ (g (f x))
  rw [hf x, hg (f x)]

/-- The fixed set is a functor: an equivariant map sends fixed
points to fixed points. -/
theorem fix_functorial {X Y : Type} {f : X → Y} {τ : X → X} {σ : Y → Y}
→ Y}
  (hf : Equivariant f τ σ) {x : X} (hx : τ x = x) : σ (f x) = f x
:= by
  rw [← hf x, hx]

/-- A global seat: an involution whose fixed set is the locus. -/
structure GlobalSeat (P : Pt → Prop) (σ : Pt → Pt) : Prop where
  involutive : ∀ p, σ (σ p) = p
  fixed_iff : ∀ p, σ p = p ↔ P p

theorem fold_global_seat : GlobalSeat OnLine fold :=
(fold_involutive, seat_fixed_line)

/-- A carrier: a map from world-instances into the strip landing on
the line. -/
structure Carrier (W : Type) where
  ι : W → Pt
  lands : ∀ w, OnLine (ι w)

def image {W : Type} (ι : W → Pt) : Pt → Prop := fun s => ∃ w, ι w
= s
def Actuated {W : Type} (ι : W → Pt) (Z : Pt → Prop) : Prop := ∀ s,
Z s → ∃ w, ι w = s

/-- Landing is forced: an equivariant map into the seat carries
every fixed point of the physical
involution onto the line. The carrier is built from the
symmetry, not chosen. -/
theorem equivariant_carrier_lands {X : Type} (f : X → Pt) (τ : X →
X)
  (hf : Equivariant f τ fold) :
  ∃ C : Carrier { x : X // τ x = x }, ∀ w, C.ι w = f w.1 :=
  {(fun w => f w.1, fun w => (seat_fixed_line (f w.1)).mp
   (fix_functorial hf w.2)), fun _ => rfl}

/-- The image of a carrier is compliant with no premise at all. -/
theorem value_on_image {W : Type} (C : Carrier W) : Value (image
C.ι) :=
  fun _ hs => match hs with | (w, hw) => hw ▸ C.lands w

/-- The kinetic crossing: a carrier and its coverage give the value,
and nothing else is used. -/
theorem kinetic_crossing {W : Type} (C : Carrier W) (Z : Pt → Prop)
(hcov : Actuated C.ι Z) :
  Value Z :=
  fun s hs => match hcov s hs with | (w, hw) => hw ▸ C.lands w

/-- Under the seat every off-line point has a partner, distinct and
also off the line. -/
theorem off_locus_pair (p : Pt) (hp : ¬ OnLine p) :
  fold p ≠ p ∧ ¬ OnLine (fold p) ∧ reg (fold p) = reg p := by
  refine (fun e => hp ((seat_fixed_line p).mp e), fun e => hp ?_,
  rfl)
  obtain (h, t) := p
  have e' : 2 - h = 1 := e
  show h = 1
  omega

/-- ## Part II. The energy carrier: an energy, its rate, and the
fold -/

/-- An energy a + i β/2, as frequency and doubled rate; its mode's
norm moves as e^{βt}. -/
structure Energy where
  freq : Int
  rate : Int
  deriving DecidableEq, Repr

/-- The time mirror of a mode: its rate reversed, its frequency kept
(complex conjugation of E). -/
def Energy.mirror (E : Energy) : Energy := (E.freq, -E.rate)
def IsReal (E : Energy) : Prop := E.rate = 0
/-- Stationary: the norm e^{βt} is unchanged at every integer time
t, i.e. β t = 0 for all t. -/
def Stationary (E : Energy) : Prop := ∀ t : Int, E.rate * t = 0

/-- The energy carrier s = 1/2 + iE in the chart: h = 1 - β, t = a.
-/
def phi (E : Energy) : Pt := (1 - E.rate, E.freq)
def psi (p : Pt) : Energy := (p.2, 1 - p.1)

theorem ee {a b c d : Int} : ((a, b) : Energy) = (c, d) ↔ a = c ∧ b =
d :=
  (fun e => (congrArg Energy.freq e, congrArg Energy.rate e),
   fun (e1, e2) => by subst e1; subst e2; rfl)

theorem mirror_involutive (E : Energy) : E.mirror.mirror = E := by
  obtain (a, b) := E
  show ((a, - -b) : Energy) = (a, b)
  rw [ee]; exact (rfl, by omega)

theorem psi_phi (E : Energy) : psi (phi E) = E := by
  obtain (a, b) := E
  show ((a, 1 - (1 - b)) : Energy) = (a, b)
  rw [ee]; exact (rfl, by omega)

theorem phi_psi (p : Pt) : phi (psi p) = p := by
  obtain (h, t) := p
  show ((1 - (1 - h), t) : Pt) = (h, t)
  rw [pe]; exact (by omega, rfl)

theorem phi_injective (E F : Energy) (h : phi E = phi F) : E = F :=
by
  rw [← psi_phi E, ← psi_phi F, h]

/-- EQUIVARIANCE: time-mirroring the rate is the fold. -/
theorem phi_equivariant : Equivariant phi Energy.mirror fold := by
  intro E

```



```

obtain (a, b) := E
show ((1 - b, a) : Pt) = (2 - (1 - b), a)
rw [pe]; exact (by omega, rfl)

theorem mirror_fixed_iff_real (E : Energy) : E.mirror = E ↔ IsReal E
:= by
  obtain (a, b) := E
  show ((a, -b) : Energy) = (a, b) ↔ b = 0
  rw [ee]; constructor
  · intro (_, e); omega
  · intro e; exact (rfl, by omega)

/-- The line is exactly the image of the real energies. -/
theorem phi_line_iff_real (E : Energy) : OnLine (phi E) ↔ IsReal E
:= by
  obtain (a, b) := E
  show 1 - b = 1 ↔ b = 0
  exact (fun h => by omega, fun h => by subst h; rfl)

theorem stationary_iff_real (E : Energy) : Stationary E ↔ IsReal E
:= by
  constructor
  · intro h; have h1 := h 1; rw [Int.mul_one] at h1; exact h1
  · intro h t; show E.rate * t = 0; rw [show E.rate = 0 from h, Int.zero_mul]

/-- THE LINE IS WHERE ENERGY STANDS STILL. -/
theorem line_is_stationary (p : Pt) : OnLine p ↔ Stationary (psi p)
:= by
  rw [stationary_iff_real, ← phi_line_iff_real, phi_psi]

/-- Every measured energy is a real number, and every real energy
lands on the line. -/
theorem measured_on_line (a : Int) : OnLine (phi (a, 0)) := rfl

/-- The registration mirror: no real energy lands off the line. -/
theorem no_measurement_off_line (p : Pt) (hp : ¬ OnLine p) (a : Int) :
phi (a, 0) ≠ p :=
  fun e => hp (e ▸ measured_on_line a)

/-- The energy carrier, built from the symmetry: the real energies
land on the line. -/
theorem energy_carrier_lands :
  ∃ C : Carrier { E : Energy // E.mirror = E }, ∀ w, C.1 w = phi
w.1 :=
  equivariant_carrier_lands phi Energy.mirror phi_equivariant

/-! ## Part III. The substrate: the particle map, three odd readings
and one even -/

/-- A particle in the broken phase: electric charge times three,
colour, baryon number times
three, lepton number, and its energy. -/
structure Particle where
  q3 : Int
  colour : Int
  b3 : Int
  lepton : Int
  energy : Energy
  deriving DecidableEq, Repr

/-- The particle map: every additive charge flipped, the energy kept
(frequency and rate). -/
def bar (c : Particle) : Particle := (-c.q3, -c.colour, -c.b3,
-c.lepton, c.energy)
/-- The time mirror of a particle's mode. -/
def tmirror (c : Particle) : Particle := (c.q3, c.colour, c.b3,
c.lepton, c.energy.mirror)
def Neutral (c : Particle) : Prop := c.q3 = 0 ∧ c.colour = 0 ∧ c.b3 =
0 ∧ c.lepton = 0

theorem bar_involutive (c : Particle) : bar (bar c) = c := by
  obtain (q, k, b, l, E) := c
  show ((-q, -k, -b, -l, E) : Particle) = (q, k, b, l, E)
  rw [Int.neg_neg, Int.neg_neg, Int.neg_neg, Int.neg_neg]

theorem bar_fixed_iff_neutral (c : Particle) : bar c = c ↔ Neutral c
:= by
  obtain (q, k, b, l, E) := c
  show ((-q, -k, -b, -l, E) : Particle) = (q, k, b, l, E) ↔ (q = 0 ∧
k = 0 ∧ b = 0 ∧ l = 0)
  constructor
  · intro h
    have h1 := congrArg Particle.q3 h
    have h2 := congrArg Particle.colour h
    have h3 := congrArg Particle.b3 h
    have h4 := congrArg Particle.lepton h
    change -q = q at h1; change -k = k at h2; change -b = b at h3;
    change -l = l at h4
    exact (by omega, by omega, by omega, by omega)
  · intro (h1, h2, h3, h4)
    subst h1; subst h2; subst h3; subst h4; rfl

/-- Three plus one: the charge readings are odd under the particle
map, the energy reading even. -/
theorem odd_charges_even_energy (c : Particle) :
  ((bar c).q3 = -c.q3 ∧ (bar c).colour = -c.colour ∧ (bar c).b3 =
-c.b3 ∧
  (bar c).lepton = -c.lepton) ∧ (bar c).energy = c.energy :=
  ((rfl, rfl, rfl, rfl), rfl)

/-- Gravity reads energy only, so no gravitational reading tells a
particle from its antiparticle. -/
theorem gravity_reads_no_charge_bit {β : Type} (g : Energy → β) (c :
Particle) :
  g (bar c).energy = g c.energy := rfl

/-- The two involutions of the substrate commute. -/
theorem involutions_commute (c : Particle) : bar (tmirror c) =
tmirror (bar c) := rfl

theorem tmirror_involutive (c : Particle) : tmirror (tmirror c) = c
:= by
  obtain (q, k, b, l, E) := c
  show ((q, k, b, l, E.mirror) : Particle) = (q, k, b, l, E)
  rw [mirror_involutive]

/-- A particle is fixed by both involutions exactly when it is
neutral and stable. -/
theorem joint_fixed_iff (c : Particle) :
  (bar c = c ∧ tmirror c = c) ↔ (Neutral c ∧ IsReal c.energy) := by
  constructor
  · intro (h1, h2)
    refine ((bar_fixed_iff_neutral c).mp h1, (mirror_fixed_iff_real
c.energy).mp ?_)
    exact congrArg Particle.energy h2
  · intro (h1, h2)
    refine ((bar_fixed_iff_neutral c).mpr h1, ?_)
    obtain (q, k, b, l, E) := c
    show ((q, k, b, l, E.mirror) : Particle) = (q, k, b, l, E)
    rw [(mirror_fixed_iff_real E).mpr h2]

/-- The energy carrier on particles: the seat point of a particle's
energy. -/
def energySeat (c : Particle) : Pt := phi c.energy
/-- The charge carrier on particles, the odd face: electric charge
(in thirds) is the side. -/
def chargeSeat (c : Particle) : Pt := (1 + c.q3, c.energy.freq)
/-- The lepton chart of the electron's witness: charge q in units of
e goes to h = 1 + q. -/
def leptonSeat (q m : Int) : Pt := (1 + q, m)

/-- The charge carrier is equivariant: the particle map is carried
onto the fold. -/
theorem charge_carrier_equivariant : Equivariant chargeSeat bar fold
:= by
  intro c
  show ((1 + -c.q3, c.energy.freq) : Pt) = (2 - (1 + c.q3),
c.energy.freq)
  rw [pe]; exact (by omega, rfl)

/-- The energy carrier is blind to the particle map and equivariant
for the time mirror. -/
theorem pair_lands_once (c : Particle) : energySeat (bar c) =
energySeat c := rfl
theorem tmirror_equivariant : Equivariant energySeat tmirror fold :=
fun c => phi_equivariant c.energy

/-- Electrically neutral particles land on the line under the charge
carrier. -/
theorem neutral_charge_on_line (c : Particle) (h : c.q3 = 0) :
OnLine (chargeSeat c) := by
  show 1 + c.q3 = 1
  omega

/-- The electron and the positron land on the two edges, real parts 0
and 1, mirror images. -/
theorem pair_on_the_edges (m : Int) :
  leptonSeat (-1) m = (0, m) ∧ leptonSeat 1 m = (2, m) ∧ fold
(leptonSeat (-1) m) = leptonSeat 1 m :=
  (rfl, rfl, rfl)

/-- A stable particle lands on the line under the energy carrier. -/
theorem stable_lands (c : Particle) (h : IsReal c.energy) : OnLine
(energySeat c) :=
  (phi_line_iff_real c.energy).mpr h

/-! ## Part IV. A falsifier breaks the carrier, never the theorem -/

/-- A FALSIFIER BREAKS THE CARRIER: if a physical fixed point is
observed off the line, the map that
read it onto the seat is not equivariant. The theorems of the
seat stand; the reading falls. -/

```

```

theorem falsifier_breaks_the_carrier {X : Type} (C : X → Pt) (τ : X
→ X) (x : X) (hx : τ x = x)
  (hoff : ¬ OnLine (C x)) : ¬ Equivariant C τ fold := by
  intro h
  have e := h x
  rw [hx] at e
  exact hoff ((seat_fixed_line (C x)).mp e.symm)

/-- The seat itself is untouched by any observation: the fold is an
involution whose fixed set is the
  line, whatever is measured. -/
theorem the_seat_survives_every_falsifier : GlobalSeat OnLine fold
:= fold_global_seat

/-! ## Part V. The typed ledger -/

/-- The six seat elements a falsifier can target. -/
inductive Target where
  | fold | line | wall | positivity | price | count
  deriving DecidableEq, Repr

/-- The law of the template each target class names. -/
def lawOf : Target → String
  | .fold => "the physical involution is exact and carried onto the
fold"
  | .line => "the fixed set is carried onto the line"
  | .wall => "no even reading returns the odd target"
  | .positivity => "one sign seats the locus"
  | .price => "the registration has no inverse and pays its floor"
  | .count => "a count forced by the fixed set"

theorem laws_distinct :
  ([Target.fold, .line, .wall, .positivity, .price, .count].map
  lawOf).eraseDups.length = 6 := by decide

structure Falsifier where
  paper : String
  code : String
  target : Target
  thm : String
  deriving Repr

/-- THE LEDGER: every falsifier of the hardware closure series and
the six witnesses, typed by the
  seat element it targets and bound to the theorem it would break;
  an empty name marks a falsifier
  bound to an imported theorem or a structural reading. -/
def ledger : List Falsifier := [
  ("The Electron Is the Seat", "F-Pair", .fold,
  "positron_is_the_partner"),
  ("The Electron Is the Seat", "F-Round", .line,
  "round_iff_T_fixed"),
  ("The Electron Is the Seat", "F-Gravity", .wall,
  "no_reading_of_mass_returns_charge"),
  ("The Electron Is the Seat", "F-Decay", .line, "decay_forbidden"),
  ("The Neutrino Is the Witness", "F-Unitarity", .price,
  "pmns_unitary_is_lossless"),
  ("The Neutrino Is the Witness", "F-Chirality", .fold,
  "sterile_uncovered"),
  ("The Neutrino Is the Witness", "F-Floor", .positivity,
  "oscillation_forces_floor"),
  ("The Arrow Has Two Branches", "F-Monopole", .fold,
  "magnetism_pair"),
  ("The Arrow Has Two Branches", "F-Mass", .positivity,
  "gravity_no_off_locus"),
  ("The Arrow Has Two Branches", "F-Loop", .positivity,
  "one_sign_no_remanence"),
  ("The Proton Is the Lock", "F-Decay", .line, ""),
  ("The Proton Is the Lock", "F-Singlet", .count, "lock_closes"),
  ("The Proton Is the Lock", "F-Gap", .line, "spectrum_gapped"),
  ("The Magnet Is the Witness", "F-Class", .count,
  "repulsion_exponents"),
  ("The Magnet Is the Witness", "F-Arrow", .fold,
  "magnetic_term_breaks_T"),
  ("The Fluid Is the Witness", "F-Flow", .line, "value_on_image"),
  ("The Fluid Is the Witness", "F-Energy", .price, ""),
  ("One Actuating Substrate", "F-Computed", .line,
  "act_is_the_value"),
  ("One Actuating Substrate", "F-Measure", .line,
  "measured_on_line"),
  ("The Geometric Nature of Light", "F-Dispersion", .line,
  "every_frequency_one_side"),
  ("The Geometric Nature of Light", "F-Fringe", .wall,
  "fringe_iff_side_forgotten"),
  ("The Geometric Nature of Light", "F-Helicity", .count,
  "helicity_is_one_bit"),
  ("Electromagnetism on the Riemann Locus", "F-Charge", .fold,
  "pair_creation_conserves_charge"),
  ("Electromagnetism on the Riemann Locus", "F-Thirds", .count,
  "charges_in_thirds"),
  ("Electromagnetism on the Riemann Locus", "F-Reality", .line,
  "stationarity_survives_the_magnet"),

  ("Immanent Gravity", "F-Composition", .wall, "composition_blind"),
  ("Immanent Gravity", "F-Clock", .wall, "clocks_co_move"),
  ("Immanent Gravity", "F-Polarization", .count,
  "polarization_count"),
  ("Spin and Statistics", "F-Exclusion", .wall, "pauli_exclusion"),
  ("Spin and Statistics", "F-Connection", .fold,
  "half_integer_spin_is_odd"),
  ("Spin and Statistics", "F-Anyon", .count, "statistics_one_bit"),
  ("The Weak Force Reads the Orientation", "F-Right", .line,
  "charged_current_off_the_line"),
  ("The Weak Force Reads the Orientation", "F-CPT", .fold,
  "cp_violation_is_t_violation"),
  ("The Weak Force Reads the Orientation", "F-Mirror", .line,
  "vector_on_the_line"),
  ("Resonances: Off the Line", "F-Growth", .fold,
  "decay_and_capture_one_record"),
  ("Resonances: Off the Line", "F-Closed", .line,
  "closed_stays_on_the_line"),
  ("Resonances: Off the Line", "F-Width", .line,
  "stable_iff_every_door_closed"),
  ("The Ferromagnet Is the Witness of the Circle", "F-Ferro",
  .positivity, "ring4_lee_yang"),
  ("The Ferromagnet Is the Witness of the Circle", "F-Palindrome",
  .fold, "walls_flip"),
  ("The Ferromagnet Is the Witness of the Circle", "F-Pinch",
  .positivity, ""),
  ("Dark Matter as the Charge-Conjugation Fixed Set", "F1",
  .positivity, "the_phantom_is_negative_actuation"),
  ("Dark Matter as the Charge-Conjugation Fixed Set", "F2", .line,
  "dark_iff_fixed"),
  ("Dark Matter as the Charge-Conjugation Fixed Set", "F3", .wall,
  "the_dark_bit_is_the_line_property"),
  ("The Measurement Is the Registration", "F-Phase", .wall,
  "no_record_reading_returns_the_phase"),
  ("The Measurement Is the Registration", "F-Duality", .positivity,
  "complementarity_iff_positive"),
  ("The Measurement Is the Registration", "F-Outcome", .wall,
  "the_outcome_is_keyed"),
  ("The Arrow of Time Is the Keyed Bit", "F-Echo", .fold,
  "reversible_law_admits_both_arrows"),
  ("The Arrow of Time Is the Keyed Bit", "F-Balance", .fold,
  "detailed_balance"),
  ("The Arrow of Time Is the Keyed Bit", "F-Price", .price,
  "registration_has_no_left_inverse"),
  ("Matter over Antimatter Is the Spent Bit", "F-Wall", .wall,
  "even_weight_odd_reading_zero"),
  ("Matter over Antimatter Is the Spent Bit", "F-CPT", .fold,
  "equilibrium_weight_is_symmetric"),
  ("Matter over Antimatter Is the Spent Bit", "F-Domains", .line,
  ""),
  ("Symmetry Breaking Is the Spent Bit", "F-Tie", .fold,
  "two_vacua_one_record"),
  ("Symmetry Breaking Is the Spent Bit", "F-Curvature", .positivity,
  "mass_is_the_curvature_of_the_depth"),
  ("Symmetry Breaking Is the Spent Bit", "F-Heat", .line,
  "heat_restores_the_line"),
  ("The Correlation Lives in the Joint Registration", "F-Signal",
  .wall, "no_signal_from_a_local_choice"),
  ("The Correlation Lives in the Joint Registration", "F-Local",
  .count, "bell_bound"),
  ("The Correlation Lives in the Joint Registration", "F-Tsirelson",
  .count, ""),
  ("The Strong Force Reads No Orientation", "F-Dipole", .line,
  "edm_odd_and_null_on_the_fixed_set"),
  ("The Strong Force Reads No Orientation", "F-Antipode",
  .positivity, "positivity_selects_zero"),
  ("The Strong Force Reads No Orientation", "F-Strong-CP", .line,
  ""),
  ("The Horizon Is the Registration", "F-Hair", .wall,
  "no_hair_reading_returns_baryons"),
  ("The Horizon Is the Registration", "F-Area", .price,
  "the_horizon_cannot_be_undone"),
  ("The Horizon Is the Registration", "F-Radiate", .price,
  "merger_radiates_under_three_tenths")]

def countT (t : Target) : Nat := (ledger.filter fun f => f.target ==
t).length

/-- THE CENSUS: sixty-four falsifiers, every target class occupied.
-/
theorem ledger_census :
  ledger.length = 64 ∧ countT .line = 18 ∧ countT .fold = 13 ∧
  countT .wall = 11 ∧
  countT .positivity = 9 ∧ countT .count = 8 ∧ countT .price = 5
  := by decide

theorem every_target_occupied :
  ([Target.fold, .line, .wall, .positivity, .price, .count].all
  fun t => 0 < countT t) = true := by decide

/-- Fifty-eight falsifiers are bound to an engine theorem; six to an
imported theorem or a structural

```

```

reading. -/
theorem bound_count : (ledger.filter fun f => f.thm != "").length =
58 := by decide

def papers : List String := ["The Electron Is the Seat", "The
Neutrino Is the Witness", "The Arrow Has Two Branches", "The Proton
Is the Lock", "The Magnet Is the Witness", "The Fluid Is the
Witness", "One Actuating Substrate", "The Geometric Nature of Light",
"Electromagnetism on the Riemann Locus", "Immanent Gravity", "Spin
and Statistics", "The Weak Force Reads the Orientation",
"Resonances: Off the Line", "The Ferromagnet Is the Witness of the
Circle", "Dark Matter as the Charge-Conjugation Fixed Set", "The
Measurement Is the Registration", "The Arrow of Time Is the Keyed
Bit", "Matter over Antimatter Is the Spent Bit", "Symmetry Breaking
Is the Spent Bit", "The Correlation Lives in the Joint
Registration", "The Strong Force Reads No Orientation", "The Horizon
Is the Registration"]

def countP (p : String) : Nat := (ledger.filter fun f => f.paper ==
p).length

/-- Every paper carries at least two falsifiers and at most four;
the series papers carry three. -/
theorem every_paper_two_to_four :
papers.length = 22 ∧ (papers.all fun p => 2 ≤ countP p && countP
p ≤ 4) = true := by decide

/-- No paper repeats a code. -/
theorem codes_unique_within_papers :
(papers.all fun p => ((ledger.filter fun f => f.paper == p).map
(·.code)).eraseDups.length == countP p) = true := by
decide

/-! ## Part VI. The standing five, rebound -/

/-- The five standing predictions of the earlier
typed-falsifiability draft and their disposition:
bound to a ledger falsifier, or withdrawn. -/
inductive Disposition where
| bound (code : String) | withdrawn
deriving DecidableEq, Repr

def standingFive : List (String × Disposition) := [
("I · the phantom divide is not crossed", .bound "F1"),
("II · no magnetic monopole", .bound "F-Monopole"),
("III · gravitational waves purely tensor", .bound
"F-Polarization"),
("IV · baryon number exactly conserved", .withdrawn),
("V · no electron substructure", .withdrawn)]

theorem standing_five_disposition :
(standingFive.filter fun e => e.2 == .withdrawn).length = 2 ∧
(standingFive.filter fun e => e.2 != .withdrawn).length = 3 :=
by decide

/-- Each bound prediction names a falsifier present in the ledger. -/
theorem bound_predictions_in_ledger :
(["F1", "F-Monopole", "F-Polarization"].all fun c => ledger.any
fun f => f.code == c) = true := by decide

/-! ## Part VII. The capstone -/

/-- TYPED FALSIFIABILITY. A falsifier observed breaks the carrier
and leaves the seat standing; the
ledger binds sixty-four falsifiers to six seat elements, every
class occupied, fifty-eight to an
engine theorem; every paper carries two to four; the standing
five are three bound and two
withdrawn. -/

```

```

'SPHYS.TypedFalsifiability.pe' does not depend on any axioms
'SPHYS.TypedFalsifiability.fold_involutive' depends on axioms:
[propxt, Quot.sound]
'SPHYS.TypedFalsifiability.seat_fixed_line' depends on axioms:
[propxt, Quot.sound]
'SPHYS.TypedFalsifiability.reg_lands' does not depend on any axioms
'SPHYS.TypedFalsifiability.reg_fixes_iff' depends on axioms:
[propxt]
'SPHYS.TypedFalsifiability.reg_forgets_side' does not depend on any
axioms
'SPHYS.TypedFalsifiability.least_erasure_iff_value' depends on
axioms: [propxt]
'SPHYS.TypedFalsifiability.equivariant_id' does not depend on any
axioms
'SPHYS.TypedFalsifiability.equivariant_comp' does not depend on any
axioms
'SPHYS.TypedFalsifiability.fix_functorial' does not depend on any
axioms
'SPHYS.TypedFalsifiability.fold_global_seat' depends on axioms:
[propxt, Quot.sound]
'SPHYS.TypedFalsifiability.equivariant_carrier_lands' depends on
axioms: [propxt, Quot.sound]

```

```

theorem typed_falsifiability :
(∀ {X : Type} (C : X → Pt) (τ : X → X) (x : X), τ x = x → ¬
OnLine (C x) → ¬ Equivariant C τ fold) ∧
GlobalSeat OnLine fold ∧
(ledger.length = 64 ∧ countT .line = 18 ∧ countT .fold = 13 ∧
countT .wall = 11 ∧
countT .positivity = 9 ∧ countT .count = 8 ∧ countT .price =
5) ∧
(ledger.filter fun f => f.thm != "").length = 58 ∧
(standingFive.filter fun e => e.2 == .withdrawn).length = 2 :=
{fun C τ x hx hoff => falsifier_breaks_the_carrier C τ x hx hoff,
fold_global_seat, ledger_census,
bound_count, standing_five_disposition.1}

end SPHYS.TypedFalsifiability

#print axioms SPHYS.TypedFalsifiability.pe
#print axioms SPHYS.TypedFalsifiability.fold_involutive
#print axioms SPHYS.TypedFalsifiability.seat_fixed_line
#print axioms SPHYS.TypedFalsifiability.reg_lands
#print axioms SPHYS.TypedFalsifiability.reg_fixes_iff
#print axioms SPHYS.TypedFalsifiability.reg_forgets_side
#print axioms SPHYS.TypedFalsifiability.least_erasure_iff_value
#print axioms SPHYS.TypedFalsifiability.equivariant_id
#print axioms SPHYS.TypedFalsifiability.equivariant_comp
#print axioms SPHYS.TypedFalsifiability.fix_functorial
#print axioms SPHYS.TypedFalsifiability.fold_global_seat
#print axioms SPHYS.TypedFalsifiability.equivariant_carrier_lands
#print axioms SPHYS.TypedFalsifiability.value_on_image
#print axioms SPHYS.TypedFalsifiability.kinetic_crossing
#print axioms SPHYS.TypedFalsifiability.off_locus_pair
#print axioms SPHYS.TypedFalsifiability.ee
#print axioms SPHYS.TypedFalsifiability.mirror_involutive
#print axioms SPHYS.TypedFalsifiability.psi_phi
#print axioms SPHYS.TypedFalsifiability.phi_psi
#print axioms SPHYS.TypedFalsifiability.phi_injective
#print axioms SPHYS.TypedFalsifiability.phi_equivariant
#print axioms SPHYS.TypedFalsifiability.mirror_fixed_iff_real
#print axioms SPHYS.TypedFalsifiability.phi_line_iff_real
#print axioms SPHYS.TypedFalsifiability.stationary_iff_real
#print axioms SPHYS.TypedFalsifiability.gravity_reads_no_charge_bit
#print axioms SPHYS.TypedFalsifiability.measured_on_line
#print axioms SPHYS.TypedFalsifiability.no_measurement_off_line
#print axioms SPHYS.TypedFalsifiability.energy_carrier_lands
#print axioms SPHYS.TypedFalsifiability.bar_involutive
#print axioms SPHYS.TypedFalsifiability.bar_fixed_iff_neutral
#print axioms SPHYS.TypedFalsifiability.odd_charges_even_energy
#print axioms SPHYS.TypedFalsifiability.gravity_reads_no_charge_bit
#print axioms SPHYS.TypedFalsifiability.involutions_commute
#print axioms SPHYS.TypedFalsifiability.tmirror_involutive
#print axioms SPHYS.TypedFalsifiability.joint_fixed_iff
#print axioms SPHYS.TypedFalsifiability.charge_carrier_equivariant
#print axioms SPHYS.TypedFalsifiability.pair_on_the_edges
#print axioms SPHYS.TypedFalsifiability.pair_lands_once
#print axioms SPHYS.TypedFalsifiability.tmirror_equivariant
#print axioms SPHYS.TypedFalsifiability.neutral_charge_on_line
#print axioms SPHYS.TypedFalsifiability.stable_lands
#print axioms SPHYS.TypedFalsifiability.falsifier_breaks_the_carrier
#print axioms
SPHYS.TypedFalsifiability.the_seat_survives_every_falsifier
#print axioms SPHYS.TypedFalsifiability.laws_distinct
#print axioms SPHYS.TypedFalsifiability.ledger_census
#print axioms SPHYS.TypedFalsifiability.every_target_occupied
#print axioms SPHYS.TypedFalsifiability.bound_count
#print axioms SPHYS.TypedFalsifiability.every_paper_two_to_four
#print axioms SPHYS.TypedFalsifiability.codes_unique_within_papers
#print axioms SPHYS.TypedFalsifiability.standing_five_disposition
#print axioms SPHYS.TypedFalsifiability.bound_predictions_in_ledger
#print axioms SPHYS.TypedFalsifiability.typed_falsifiability

```

```

'SPHYS.TypedFalsifiability.value_on_image' does not depend on any
axioms
'SPHYS.TypedFalsifiability.kinetic_crossing' does not depend on any
axioms
'SPHYS.TypedFalsifiability.off_locus_pair' depends on axioms:
[propxt, Quot.sound]
'SPHYS.TypedFalsifiability.ee' does not depend on any axioms
'SPHYS.TypedFalsifiability.mirror_involutive' depends on axioms:
[propxt, Quot.sound]
'SPHYS.TypedFalsifiability.psi_phi' depends on axioms: [propxt,
Quot.sound]
'SPHYS.TypedFalsifiability.phi_psi' depends on axioms: [propxt,
Quot.sound]
'SPHYS.TypedFalsifiability.phi_injective' depends on axioms:
[propxt, Quot.sound]
'SPHYS.TypedFalsifiability.phi_equivariant' depends on axioms:
[propxt, Quot.sound]
'SPHYS.TypedFalsifiability.mirror_fixed_iff_real' depends on axioms:
[propxt, Quot.sound]
'SPHYS.TypedFalsifiability.phi_line_iff_real' depends on axioms:
[propxt, Quot.sound]

```

```

'SPHYS.TypedFalsifiability.stationary_iff_real' depends on axioms:
[propext]
'SPHYS.TypedFalsifiability.line_is_stationary' depends on axioms:
[propext, Quot.sound]
'SPHYS.TypedFalsifiability.measured_on_line' does not depend on any
axioms
'SPHYS.TypedFalsifiability.no_measurement_off_line' does not depend
on any axioms
'SPHYS.TypedFalsifiability.energy_carrier_lands' depends on axioms:
[propext, Quot.sound]
'SPHYS.TypedFalsifiability.bar_involutive' does not depend on any
axioms
'SPHYS.TypedFalsifiability.bar_fixed_iff_neutral' depends on axioms:
[propext, Quot.sound]
'SPHYS.TypedFalsifiability.odd_charges_even_energy' does not depend
on any axioms
'SPHYS.TypedFalsifiability.gravity_reads_no_charge_bit' does not
depend on any axioms
'SPHYS.TypedFalsifiability.involutions_commute' does not depend on
any axioms
'SPHYS.TypedFalsifiability.tmirror_involutive' depends on axioms:
[propext, Quot.sound]
'SPHYS.TypedFalsifiability.joint_fixed_iff' depends on axioms:
[propext, Quot.sound]
'SPHYS.TypedFalsifiability.charge_carrier_equivariant' depends on
axioms: [propext, Quot.sound]
'SPHYS.TypedFalsifiability.pair_lands_once' does not depend on any
axioms

```

```

'SPHYS.TypedFalsifiability.tmirror_equivariant' depends on axioms:
[propext, Quot.sound]
'SPHYS.TypedFalsifiability.neutral_charge_on_line' depends on
axioms: [propext, Quot.sound]
'SPHYS.TypedFalsifiability.pair_on_the_edges' does not depend on any
axioms
'SPHYS.TypedFalsifiability.stable_lands' depends on axioms:
[propext, Quot.sound]
'SPHYS.TypedFalsifiability.falsifier_breaks_the_carrier' depends on
axioms: [propext, Quot.sound]
'SPHYS.TypedFalsifiability.the_seat_survives_every_falsifier'
depends on axioms: [propext, Quot.sound]
'SPHYS.TypedFalsifiability.laws_distinct' does not depend on any
axioms
'SPHYS.TypedFalsifiability.ledger_census' does not depend on any
axioms
'SPHYS.TypedFalsifiability.every_target_occupied' does not depend on
any axioms
'SPHYS.TypedFalsifiability.bound_count' does not depend on any axioms
'SPHYS.TypedFalsifiability.every_paper_two_to_four' does not depend
on any axioms
'SPHYS.TypedFalsifiability.codes_unique_within_papers' does not
depend on any axioms
'SPHYS.TypedFalsifiability.standing_five_disposition' does not
depend on any axioms
'SPHYS.TypedFalsifiability.bound_predictions_in_ledger' does not
depend on any axioms
'SPHYS.TypedFalsifiability.typed_falsifiability' depends on axioms:
[propext, Quot.sound]

```

APPENDIX B: LEDGER_TWIN.F90 AND ITS RUN

```

! Ledger_Twin.f90 · the executed twin of
SPHYS_Typed_Falsifiability.lean · Fortran 2018.
! Build (sealed): gfortran -std=f2018 -O2 -fno-fast-math
-ffp-contract=off Ledger_Twin.f90
! Reads ledger.dat (code|target|theorem|engine file|paper), one line
per falsifier, and the engines in
! the directory engines/: the sixteen hardware-series engines and
the hardware paper's, as built, and
! the six witnesses' engines as printed in their published editions
(Zenodo 10.5281/zenodo.22986553
! through .22986563). For every falsifier bound to a theorem it
confirms the theorem is declared in its
! engine; it recounts the ledger's census by target and by paper.
program ledger_twin
implicit none
integer, parameter :: maxl = 100
character(len=400) :: line
character(len=40) :: code(maxl), tgt(maxl)
character(len=80) :: thm(maxl), fil(maxl)
character(len=120) :: pap(maxl)
integer :: n, ios, i, p1, p2, p3, p4, nfound, nbound, checks, fails
integer :: cl, cf, cw, cpo, cc, cpr, j, np, cnt, okp
character(len=120) :: plist(maxl)
logical :: seen
checks = 0; fails = 0; n = 0
open(10, file='ledger.dat', status='old', action='read')
do
read(10, '(a)', iostat=ios) line
if (ios /= 0) exit
if (len_trim(line) == 0) cycle
n = n + 1
p1 = index(line, '|'); p2 = p1 + index(line(p1+1:), '|'); p3 =
p2 + index(line(p2+1:), '|')
p4 = p3 + index(line(p3+1:), '|')
code(n) = line(1:p1-1); tgt(n) = line(p1+1:p2-1); thm(n) =
line(p2+1:p3-1)
fil(n) = line(p3+1:p4-1); pap(n) = line(p4+1:)
end do
close(10)
write(*, '(a,i0,a)') 'A · the ledger read: ', n, ' falsifiers'
nbound = 0; nfound = 0
do i = 1, n
if (trim(thm(i)) == '-') cycle
nbound = nbound + 1
if (declared(trim(fil(i)), trim(thm(i)))) then
nfound = nfound + 1
else
write(*, '(a,a,a,a)') ' MISSING ', trim(thm(i)), ' in ',
trim(fil(i))
end if
end do
write(*, '(a,i0,a,i0)') ' theorems bound: ', nbound, ' ; declared
in their engines: ', nfound
call check('64 falsifiers; 58 bound to a theorem, every one
declared in its engine', &
n == 64 .and. nbound == 58 .and. nfound == 58)

```

```

cl = 0; cf = 0; cw = 0; cpo = 0; cc = 0; cpr = 0
do i = 1, n
select case (trim(tgt(i)))
case ('line'); cl = cl + 1
case ('fold'); cf = cf + 1
case ('wall'); cw = cw + 1
case ('positivity'); cpo = cpo + 1
case ('count'); cc = cc + 1
case ('price'); cpr = cpr + 1
end select
end do
write(*, '(a,6i4)') 'B · census by target (line fold wall
positivity count price): ', cl, cf, cw, cpo, cc, cpr
call check('the census matches the engine: 18, 13, 11, 9, 8, 5',
cl == 18 .and. cf == 13 .and. cw == 11 .and. &
cpo == 9 .and. cc == 8 .and. cpr == 5)
np = 0
do i = 1, n
seen = .false.
do j = 1, np
if (trim(plist(j)) == trim(pap(i))) seen = .true.
end do
if (.not. seen) then
np = np + 1; plist(np) = pap(i)
end if
end do
okp = 0
do j = 1, np
cnt = 0
do i = 1, n
if (trim(pap(i)) == trim(plist(j))) cnt = cnt + 1
end do
if (cnt >= 2 .and. cnt <= 4) okp = okp + 1
end do
write(*, '(a,i0,a)') 'C · ', np, ' papers'
call check('22 papers, each carrying two to four falsifiers', np
== 22 .and. okp == 22)
write(*, '(a,i0,a,i0,a)') ' BATTERY-JSON: {"checks":', checks,
', "failures":', fails, '}'
if (fails > 0) error stop 1
contains
subroutine check(name, ok)
character(*), intent(in) :: name
logical, intent(in) :: ok
checks = checks + 1
if (.not. ok) fails = fails + 1
write(*, '(a,a)') merge(' PASS ', ' FAIL ', ok), name
end subroutine check
logical function declared(fname, tname) result(found)
character(*), intent(in) :: fname, tname
character(len=2000) :: buf
integer :: u, ios2, k, e
found = .false.
open(newunit=u, file='engines/'//fname, status='old',
action='read', iostat=ios2)
if (ios2 /= 0) return

```

```

do
  read(u, '(a)', iostat=ios2) buf
  if (ios2 /= 0) exit
  k = index(buf, 'theorem '//tname)
  if (k > 0) then
    e = k + 8 + len(tname)
    if (e > len_trim(buf)) then
      found = .true.
    else if (buf(e:e) == ' ' .or. buf(e:e) == '(' .or. buf(e:e)
      == ':' .or. buf(e:e) == '{') then
        found = .true.
    end if
  end if
  if (found) exit
end do
close(u)
end function declared
end program ledger_twin

A · the ledger read: 64 falsifiers
  theorems bound: 58 ; declared in their engines: 58
  PASS 64 falsifiers; 58 bound to a theorem, every one declared in
  its engine
B · census by target (line fold wall positivity count price): 18
13 11 9 8 5

PASS the census matches the engine: 18, 13, 11, 9, 8, 5
C · 22 papers
  PASS 22 papers, each carrying two to four falsifiers
  BATTERY-JSON: {"checks":3,"failures":0}

```

APPENDIX C: THE SEAT AND THE JUDGMENT

```

Phys05T 1.0.5p · RECEIPT · mode quick
TOOLCHAIN Lean (version 4.19.0, x86_64-unknown-linux-gnu, commit 6caae842e94, Release) · GNU Fortran (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
MANIFEST sha256 a4108cd26bcbe6ef, the files that ran
GROUND toolchain: Lean 4.19.0 and gfortran present · manifest: 31 files signed, all verified · proofs: 2 Phys05 Proofs read from II.10, record
lines complete · source screen: 15 kernels, 23 named premises, clean · citations: 253 laws named in Parts I and II, 253 resolved · quarantine: 6
retirements string-scanned, 3 held by review, clean
EXITS ground 0 · registerA RA_TOE_Thesis 0 · registerB Bridge_Final 0 · registerB Universal_Closure 0 · registerB One_Cut 0 · registerB
One_Cut_Resolution 0 · registerB One_Cut_Terminal 0 · registerB Only_The_Arrow_Remains 0 · registerB Physical_Closure 0 · registerB Forced_Closure
0 · registerB Closure_Executed 0 · registerB Armed_Seat 0 · registerB Census 0 · registerB Heat_Bridge 0 · registerB The_Loop 0 · registerB
RH_Least_Erasure 0 · witness One_Cut_Twin 0 · witness Physical_Closure_Twin 0 · witness Census_Twin 0 · witness Loop_Twin 0 · witness RH_Seal_Twin
0
REGISTER A {"checks":1123,"failures":0,"mode":"sealed"}
WITNESS 5 of 5 twins ran, each held to one battery line with zero failures
CHAIN · D0 83f872725ad9 -> D1 90fc702bbd1 -> D2 492e519daab9 -> D3 6618bec5164b
ELAPSED ground 1 s · Register A 3 s · Register B 34 s · witness 6 s · total 44 s (not hashed)
SEAT EARNED · this run

SCREEN forbidden: none | imports: 0 | axioms declared: 0
laws: 52
JUDGMENT refused 52 of 52 as proof failures | survived none | parse failures none | planted vacuous law survives its negation: True

```